

Design and Synthesis of 4-Bit Counter Using Yosys

INTRODUCTION:

In this lab, students will learn to synthesize 4-Bit Counter using Yosys. They will practice reading the design, performing optimization, mapping it to a standard cell library, and generating the gate-level netlist. This exercise demonstrates how RTL code is transformed into hardware-efficient logic.

APPLICATION:

Synchronous 4-bit counters are used in frequency division, digital clocks, timers, and event counting. They also serve as building blocks for registers, sequence generators, and other complex digital circuits.

OBJECTIVE:

After completing this lab, you will be able to:

- Write Verilog code for the design
- Load the RTL design into Yosys for synthesis.
- Perform synthesis to map the design to the target standard cell library
- Understand mapping of the design to standard cells.
- Generate the gate-level netlist and understand its structure.

PROCEDURE TO CREATE A LAB:

Step 1: Set up Project Directory

- 1.1: Create a directory for your lab: `mkdir <project_name>`
- 1.2: Open lab directory: `cd <Project_name>`
- 1.3: Create directory for project: `mkdir counter`
- 1.4: Open project directory: `cd counter`

Step 2: Create Design file

- 2.1: Open gvim editor for design with command: `gvim counter.v`

RTL Design Code:

```
module counter_4bit (  
    input clk,  
    input reset,  
    output reg [3:0] count );  
  
    always @(posedge clk or posedge reset) begin  
        if (reset)  
            count <= 4'b0000;  
        else  
            count <= count + 1; // Increment counter  
            // Reset counter to 0  
    end  
endmodule
```

2.2: Save the file using command- :wq!

Step 3: Synthesize the Design with Yosys

Invoke Yosys in a new terminal by the following command: yosys

3.1: Use the following command to Synthesize:

```
# Read the Verilog source file  
read_verilog counter.v  
  
# Set the top-level module and check hierarchy  
hierarchy -check -top counter_4bit  
  
# high level stuff  
proc; opt; fsm; opt; memory; opt  
  
# mapping to internal cell library  
techmap; opt  
  
# mapping flip-flops to mycells.lib  
dfflibmap -liberty osu018_stdcells.lib
```

```
# mapping logic to mycells.lib
abc -liberty osu018_stdcells.lib

# Clean up unused cells and signals after mapping
clean

# Write the final synthesized gate-level netlist to a new Verilog file
write_verilog counter4bit_synth.v

#Generate a graphical visualization of the synthesized circuit
Show
```

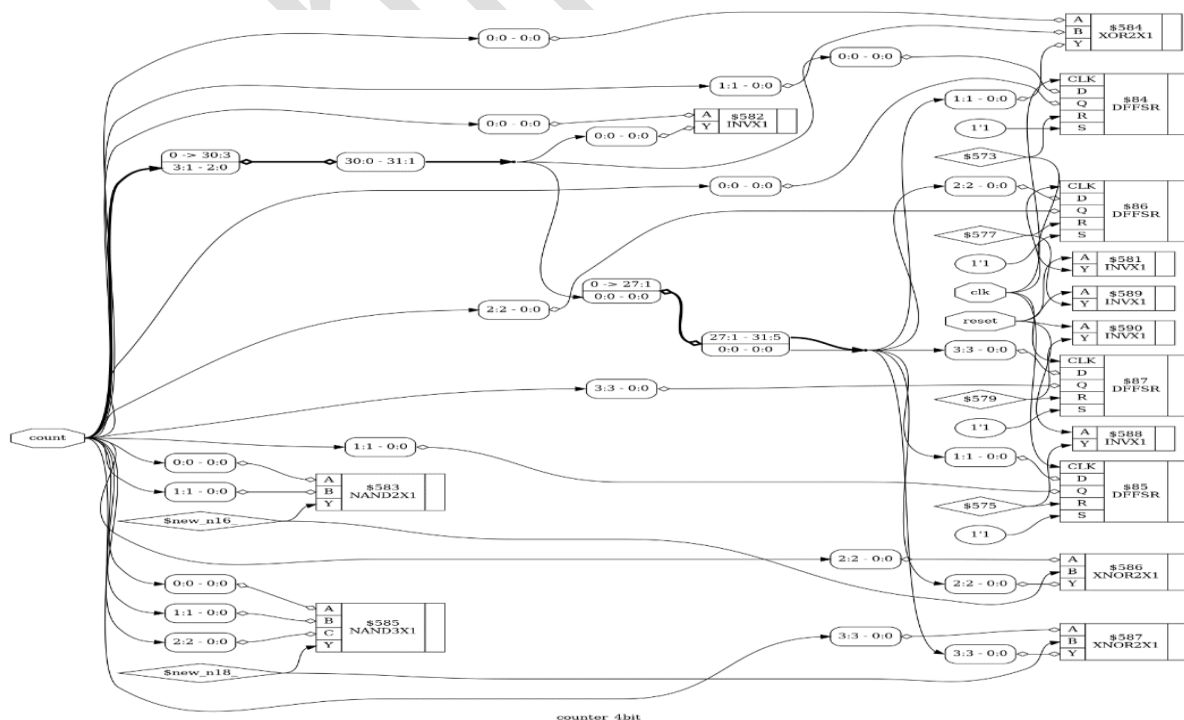
Step 4: View Output File (Netlist)

```
gvim counter4bit_synth.v
```

OBSERVATION:

- The design is synthesized without errors.
- Flip-flops are used for storing the counter state.
- The increment operation is mapped into combinational logic.
- Reset logic is correctly synthesized to initialize the counter to 0000.

OUTPUT:



CONCLUSION:

The Yosys synthesis output confirms the correct implementation of the 4-bit synchronous up counter. The RTL design is successfully read, optimized, and mapped to the target standard cell library. The generated gate-level netlist shows that the counter logic is preserved, with flip-flops used to store the state and combinational logic added for the increment operation. The synthesis report verifies that the reset logic is properly integrated to initialize the counter to 0000. The design hierarchy, connectivity, and logic mapping results in Yosys match the intended functionality of the counter, ensuring that the hardware can reliably perform binary counting from 0000 to 1111 and then roll over.